# ATmega128 project 주제 정하기

project

2025/05/26 11:37 http://blog.naver.com/1225seong/223877953191

좋습니다! 이미 꽤 많은 하드웨어 경험을 쌓으셨네요.

- **LED 제어**,
- **키 매트릭스 입력**,
- **ADC**,
- **7-Segment**,
- **DC 모터 제어**,
- **UART 통신**까지 다뤄보셨다면, 이제는 **기능 통합**, **상태 기반 제어(FSM)**, **인터페이스 응용**, **모듈화된 설계** 위주로 심화 프로젝트를 추천드릴 수 있습니다.

---

**? 추천 프로젝트 (심화/통합형)**

**1. ? 스마트 교통 신호 시스템 시뮬레이터**

FSM + LED + Key Input + 7-Segment + UART

- **기능**
- 교차로 신호 FSM 설계 (보행자 버튼 입력에 따른 신호 변경)
- 남은 시간 7세그먼트에 표시
- UART로 현재 상태 실시간 출력 (디버깅 또는 관제용)
- **학습 포인트**
- 상태머신(FSM) 설계
- 멀티타이머 제어
- UART 디버깅 활용
- 실시간 입출력 통합 구조 설계

---

**2. ? 간이 자동 분류 시스템 (물류 컨베이어 시뮬레이션)**

ADC + DC 모터 + Keypad + 7-Segment

- **기능**
- ADC로 측정된 물체 정보 (예: 무게 센서 또는 조도 기반)
- 일정 기준 이상이면 컨베이어 작동 (모터 구동), 카운팅 표시

- Keypad로 분류 기준 설정, UART로 실시간 로그
- **학습 포인트**
- 조건 기반 제어
- 사용자 설정값 반영
- 센서-모터-UI의 통합

---

### 3. ? 간단한 기억력 테스트 게임기

LED + Key Matrix + 7Segment + UART

- **기능**
- 무작위 LED 시퀀스를 표시 → 사용자가 키 매트릭스로 순서 입력
- 정답 여부에 따라 점수 표시 (7-segment)
- UART로 로그 출력
- **학습 포인트**
- 배열 기반 상태 추적
- 입력 디바운싱과 빠른 응답 처리
- 간단한 게임 FSM 설계

---

### 4. ? UART 기반 센서 모니터링 대시보드 (터미널 기반)

ADC + UART + 키 입력

- **기능**
- 실시간 ADC 센서값 UART로 출력
- 키 입력으로 센서 채널 선택, 범위 지정 등 인터랙션
- PC 터미널(예: RealTerm, TeraTerm)과 인터페이스
- **학습 포인트**
- UART RX 수신 FSM 설계
- 문자열 처리 (ASCII → 정수)
- 간단한 CLI 구조 흉내내기

---

### 5. ? PWM 기반 DC 모터 속도 제어기

ADC + DC 모터 + 키 입력 + 7-Segment

- **기능**
- ADC 값으로 속도 조절 (가변저항 사용)

- 키로 방향 전환 (정회전/역회전)
- 현재 속도 단계 7-Segment로 표시
- **학습 포인트**
- PWM 응용 (속도 제어)
- 방향 제어를 위한 로직
- 사용자 인터페이스 통합

---

**? 학습 확장 포인트**

| 주제 | 확장 아이디어 |
|---|---|
| FSM | LED 패턴, 게임 시스템, 자동제어 로직 |
| UART | 명령 파싱 → 명령어 기반 시스템 제어 |
| 모터 | 속도/방향 제어 → 피드백 제어 실험으로 확장 |
| ADC | 여러 센서 조합 → 조건 기반 제어 로직 |
| 키 입력 | 메뉴 UI 구현, 간단한 인터페이스 시스템 |

---

**? 참고: 상태 기반 설계 예시 (FSM 구조)**

typedef enum {IDLE, LED_ON, LED_OFF} State; State current = IDLE; void loop() { switch(current) { case IDLE: if (button_pressed) current = LED_ON; break; case LED_ON: LED = 1; delay(1000); current = LED_OFF; break; case LED_OFF: LED = 0; delay(1000); current = IDLE; break; } }

---

**? 질문**

- 위 프로젝트 중 어떤 방향이 가장 흥미로우신가요?
- 혹은 '이런 기능을 꼭 넣고 싶은데 어떻게 조합하면 좋을까?' 하는 아이디어 있으신가요?

원하시는 방향을 알려주시면, 구체적인 회로도/코드 틀/설계 로직까지 도와드릴 수 있습니다.

# 6/2 ATmega128: 프로젝트

project

2025/06/02 16:17 http://blog.naver.com/1225seong/223886209371

1. 시스템 명세서
2. Use-case
3. System-Block
4. Coding
5. Test

**프로젝트 : 스마트 주차 시스템**

1. 명세서

1번 아두이노 보드master 서버 관리 역할
2번, 3번 slave 주차장 A, B 역할

주차장에 차량이 들어오는 것을 초음파 센서로 감지함. DC motor로 차단기 open 구동. 나갈 때도 마찬가지
또는 야간이라 가정하면 불 비추었을 때도 차량이 들어오는 것으로 간주 // 이건 선택사항
주차 대수는 총 16대.
마스터에서 입차시 부터 로그 기록. 입출차시간 및
출차 시 사용시간에 따라 요금 부과. 요금을 LCD에 띄워줌
해당 요금을 키패드에 입력하면 정산된 것으로 간주하고 차단기 open
무단 출차시 부저로 경보음 울리기, LCD에 경고 메세지 출력

---

**? 확장 아이디어 (선택 사항)**

| 추가 기능 | 설명 |
| --- | --- |
| RFID | 차량 자동 인식, 자동 정산 |
| SD 카드 저장 | 입출차 기록, 요금 로그 백업 |

| | |
|---|---|
| **무선 연동 (ESP8266)** | 웹서버 연동, 관리자 실시간 모니터링 |
| **관리자 모드** | 키패드로 수동 제어 및 초기화 기능 |
| **통신 방식** | UART방식 해보고 나중에 I2C 방식도 해보기 |

---

스마트 주차 관리 시스템

Ⅰ. 명세서 (Requirement Specification)

1. 프로젝트 개요 (Overview)

| 항목 | 내용 |
|---|---|
| 1.1 프로젝트명 | 스마트 주차 정산 시스템 |
| 1.2 배경 및 목적 | 다수의 주차장 데이터를 통합 관리하고, 실시간 주차 현황과 요금 정산을 자동화하여 편의성과 효율성을 높이기 위함 |
| 1.3 범위 | 두 개의 주차장(보드 2개)을 슬레이브로 하여 입출차 상태를 감지하고, 마스터 보드에서 입출차 기록 및 요금 정산을 수행함 |

2. 시스템 요구사항

2.1 기능 요구사항 (Functional)

1. 차량의 입차 및 출차 감지 기능 (초음파 센서를 통해)

2. 차량 ID 또는 고유번호 입력 및 인식

3. 입출차 시간 기록 및 주차 요금 정산 기능

4. LCD를 통한 실시간 주차 현황 표시

5. 마스터-슬레이브 간 UART 기반 통신

2.2 비기능 요구사항 (Non-Functional)

- 성능: 응답 시간 ≤ 300 ms

- 신뢰성: 통신 실패 시 재시도 3회 후 오류 표시

- 보안: 차량 ID는 EEPROM에 암호화 저장 가능하도록 구조 고려

- 확장성: 보드 수가 늘어나도 통신 프로토콜 변경 없이 확장 가능

### 3. 시스템 구성도 및 아키텍처

- 하드웨어 구성
- ATmega128 × 3, 초음파 센서 × 4, LCD × 3, Keypad × 2, 부저 × 1

- 소프트웨어 계층
- • 응용 층 ↔ 통신 층 ↔ 드라이버 층

블록 다이어그램

[Master MCU]?UART?[Slave A MCU]
?UART?[Slave B MCU]

### 4. 인터페이스 정의 (Interface Specification)

| 구분 | 항목 | 규격 |
|---|---|---|
| UART | Baud Rate | 9600 bps |
| | 데이터 포맷 | 8-N-1 |
| | 프로토콜 | STX - [Payload] - ETX - CRC |
| 센서 | 초음파 TRIG | PD2 |

초음파 ECHOPD3

5. 제약사항 (Constraints)

- MCU Flash 메모리 8 KB 이내 사용
- 외부 전원 5 V ± 0.2 V
- LCD는 16x2 사용, 문자 오버플로우 방지 고려
- 초음파 센서 간 간섭 방지 위해 일정 시간 딜레이 적용

6. 테스트 계획 요약

| 테스트 ID | 기능 | 입력 | 기대 결과 |
|---|---|---|---|
| T-01 | 입차 감지 | 차량 모형 투입 | LCD에 "주차 중" 표시 |
| T-02 | 출차 및 요금 정산 | 차량 ID OUT 입력 | 입출차 시간 표시 및 요금 표시 |
| T-03 | 주차 현황 표시 | 입출차 반복 | 빈 자리 수 LCD 실시간 업데이트 |

7. 용어 정의 (Glossary)

| 용어 | 정의 |
|---|---|
| ID | 차량 번호 또는 고유 식별자 |
| IN/OUT | 입차/출차 이벤트 구분 |
| Slave A/B | 각각 주차장 A와 B 담당 보드 |
| Master | 입출차 기록 및 요금 정산을 담당하는 중앙 보드 |

---

II . Use-Case

UC-01 차량 입차 처리

| 항목 | 내용 |
| --- | --- |
| Use Case 이름 | UC-01 차량 입차 처리 |
| 주요 액터 | 차량 사용자, Slave A MCU, Master MCU |
| 사전 조건 | 시스템 부팅 완료, 센서 대기 상태 |
| 후조건 | 입차 기록 저장, LCD 업데이트 |
| 기본 흐름 | 1) 초음파 센서 차량 감지 → 2) 차량 ID 입력 (Keypad) → 3) Slave → Master 전송 → 4) 시간 저장 |
| 예외 흐름 | 차량 ID 중복 또는 입력 누락 시 재입력 요청 표시 |
| 관련 시스템 | 초음파 센서, LCD, UART 통신, Master RTC 기능 |

UC-02 차량 출차 처리

| 항목 | 내용 |
| --- | --- |
| Use Case 이름 | UC-02 차량 출차 처리 |
| 주요 액터 | 차량 사용자, Slave A MCU, Master MCU |
| 사전 조건 | 차량 ID가 이미 시스템에 등록되어 있어야 함 |
| 후조건 | 출차 시간 기록, 주차 요금 계산 및 표시 |
| 기본 흐름 | 1) 초음파로 차량 출차 감지 → 2) 차량 ID 확인 → 3) Master로 전송 → 4) 시간 차이 계산 및 요금 표시 |
| 예외 흐름 | ID 불일치 시 LCD에 오류 메시지 표시 |
| 관련 시스템 | RTC, LCD, UART, EEPROM |

UC-03 주차 현황 표시

| 항목 | 내용 |
| --- | --- |
| Use Case 이름 | UC-03 주차 현황 표시 |
| 주요 액터 | 사용자, Slave MCU |
| 사전 조건 | 시스템 정상 구동 중 |

| | |
|---|---|
| 후조건 | LCD에 현재 빈 자리/주차된 자리 수 표시 (최대 8대) |
| 기본 흐름 | 1) 입출차 감지 이벤트 발생 시 → 2) 주차 공간 수 계산 → 3) LCD 업데이트 |
| 예외 흐름 | 센서 고장 시 오류 메시지 표시 |
| 관련 시스템 | 초음파 센서, LCD |

UC-04 요금 정산

| 항목 | 내용 |
|---|---|
| Use Case 이름 | UC-04 요금 정산 |
| 주요 액터 | Master MCU, 사용자 |
| 사전 조건 | 출차 이벤트 발생, ID가 기록되어 있어야 함 |
| 후조건 | 요금 계산 후 LCD에 표시되며, EEPROM에 기록됨 |
| 기본 흐름 | 1) 출차 시 ID 확인 → 2) 입출차 시간 계산 → 3) 요금 계산 → 4) LCD 표시 및 기록 |
| 예외 흐름 | 시간 계산 실패 또는 RTC 오류 시 기본 요금 부과 |
| 관련 시스템 | RTC, EEPROM, LCD, UART |

---

**오늘 할 것**

master - slave 간 UART 통신 프로토콜 설계

/* * Master_UART.c * * Created: 2025-06-02 오후 3:36:33 * Author : me */ #include <avr/io.h> #include <util/delay.h> #define F_CPU 16000000L void UART_init(void) { UBRR1H = 0; UBRR1L = 103; UCSR1B = (1 << RXEN1) | (1 << TXEN1); UCSR1C = (1 << UCSZ11) | (1 << UCSZ10); } void UART_transmit(uint8_t data) { while (!(UCSR1A & (1 << UDRE1))); UDR1 = data; } uint8_t UART_receive(void) { while (!(UCSR1A & (1 << RXC1))); return UDR1; } int main(void) { UART_init(); // LED 연결용 포트 예시: PORTA0 (핀 40번) DDRA |= (1 << PA0); // PA0을 출력으로 설정 while (1) { UART_transmit('M'); _delay_ms(1000); uint8_t data = UART_receive(); if (data == 'S') { PORTA |= (1 << PA0); // 수신 성공: LED ON _delay_ms(500); PORTA &= ~(1 << PA0); // LED OFF } } } /* * Slave_UART.c.c * * Created: 2025-06-02 오후 3:41:14 * Author : me */ #include <avr/io.h> #include <util/delay.h> void UART_init(void) { UBRR1H = 0; UBRR1L = 103; // 16MHz 기준, 9600bps (일반모드) UCSR1B = (1 << RXEN1) | (1 << TXEN1); // 송수신 활성화 UCSR1C = (1 << UCSZ11) | (1 << UCSZ10); // 8-bit 데이터 } void UART_transmit(uint8_t data) { while (!(UCSR1A & (1 << UDRE1))); // 송신 가능 대기 UDR1 = data; }

uint8_t UART_receive(void) { while (!(UCSR1A & (1 << RXC1))); // 수신 완료 대기 return UDR1; } int main(void) { UART_init(); while (1) { uint8_t data = UART_receive(); // 마스터로부터 수신 if (data == 'M') { UART_transmit('S'); // 'S' 응답 전송 } } }

led 전원 on 확인

# ATmega128 프로젝트 : 스마트 주차 시스템 ; slave A 입차 시스템

project

2025/06/10 14:32 http://blog.naver.com/1225seong/223894567440

/* slave A (입차) 구현해야 할 기능 1. 입차 시 초음파 센서에 물체 감지 -> 외부 인터럽트 발동 lcd에 "차량번호 입력" 띄우기 2. 차량 번호 입력하는 키보드 구현 -> key matrix로 3. 입력 시 7segment에 차량번호 띄우기. 4. enter 입력하면 master로 프로토콜 전송 >> 차량번호만 보냄 // master에서 차량번호 받은 순간부터 시간 잼 5. 주차 공간 가능 대수 master로 부터 uart로 전송받아서 띄우기 6. 현재 남아있는 주차공간 대수 led로 표현 (총 8자리. 차면 하나씩 꺼짐) 7. master로 전송받은 주차 가능 공간이 만석이면 led에 "만차" 띄우기 */ #define F_CPU 16000000UL #include <avr/io.h> #include <avr/interrupt.h> #include <util/delay.h> //////LCD 출력 관련 포트 연결///// #define PORT_DATA PORTA // 데이터 핀 연결 포트 A #define PORT_CONTROL PORTG // 제어 핀 연결 포트 G #define DDR_DATA DDRA // 데이터 핀의 데이터 방향 #define DDR_CONTROL DDRG // 제어 핀의 데이터 방향 #define RS_PIN 0 // RS 제어 핀의 비트 번호 #define RW_PIN 1 // R/W 제어 핀의 비트 번호 #define E_PIN 2 // E 제어 핀의 비트 번호 #define COMMAND_CLEAR_DISPLAY 0x01 #define COMMAND_8_BIT_MODE 0x38 // 8비트, 2라인, 5x8 폰트 #define COMMAND_4_BIT_MODE 0x28 // 4비트, 2라인, 5x8 폰트 #define COMMAND_DISPLAY_ON_OFF_BIT 2 #define COMMAND_CURSOR_ON_OFF_BIT 1 #define COMMAND_BLINK_ON_OFF_BIT 0 /// 7segment //// uint8_t numbers[] = {0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xd8, 0x80, 0x98, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e, 0x7f}; uint8_t sel[] = {0b1110, 0b1101, 0b1011, 0b0111}; /// 키패드 입력 KEY MATRIX /// #define Key_data_input PINF #define Key_data_output PORTF /// LCD //// void LCD_pulse_enable(void) // 하강 에지에서 동작 { PORT_CONTROL |= (1 << E_PIN); // E를 HIGH로 _delay_us(1); PORT_CONTROL &= ~(1 << E_PIN); // E를 LOW로 _delay_ms(1); } void LCD_write_data(uint8_t data) { PORT_CONTROL |= (1 << RS_PIN); // 문자 출력에서 RS는 1 PORT_DATA = data; // 출력할 문자 데이터 LCD_pulse_enable(); // 문자 출력 _delay_ms(2); } void LCD_write_command(uint8_t command) { PORT_CONTROL &= ~(1 << RS_PIN); // 명령어 실행에서 RS는 0 PORT_DATA = command; // 데이터 핀에 명령어 전달 LCD_pulse_enable(); // 명령어 실행 _delay_ms(2); } void LCD_clear(void) { LCD_write_command(COMMAND_CLEAR_DISPLAY); _delay_ms(2); } void LCD_init(void) { _delay_ms(50); // 초기 구동 시간 // 연결 핀을 출력으로 설정 DDR_DATA = 0xFF; PORT_DATA = 0x00; DDR_CONTROL |= (1 << RS_PIN) | (1 << RW_PIN) | (1 << E_PIN); // RW 핀으로 LOW를 출력하여 쓰기 전용으로 사용 PORT_CONTROL &= ~(1 << RW_PIN); LCD_write_command(COMMAND_8_BIT_MODE); // 8비트 모드 // display on/off control // 화면 on, 커서 off, 커서 깜빡임 off uint8_t command = 0x08 | (1 << COMMAND_DISPLAY_ON_OFF_BIT); LCD_write_command(command); LCD_clear(); // 화면 지움 // Entry Mode Set // 출력 후 커서를 오른쪽으로 옮김, 즉, DDRAM의 주소가 증가하며 화면 이동은 없음 LCD_write_command(0x06); } void LCD_write_string(char *string) { uint8_t i; for(i = 0; string[i]; i++) // 종료 문자를 만날 때까지 LCD_write_data(string[i]); // 문자 단위 출력

} void LCD_goto_XY(uint8_t row, uint8_t col) { col %= 16; // [0 15] row %= 2; // [0 1] // 첫째 라인 시작 주소는 0x00, 둘째 라인 시작 주소는 0x40 uint8_t address = (0x40 * row) + col; uint8_t command = 0x80 + address; LCD_write_command(command); // 커서 이동 } ///초음파 함수 ///// // 초음파 측정용 변수 volatile uint16_t timer_start = 0; volatile uint16_t timer_end = 0; volatile uint8_t echo_flag = 0; // 초음파 센서 핀 정의 #define TRIG_PIN PE4 #define ECHO_PIN PE5 // ---------- 초음파 함수 ---------- void ultrasonic_init(void) { DDRE |= (1 << TRIG_PIN); // TRIG 출력 DDRE &= ~(1 << ECHO_PIN); // ECHO 입력 // INT5 인터럽트 설정 (상승 에지 감지) EICRB |= (1 << ISC51) | (1 << ISC50); EIMSK |= (1 << INT5); // 타이머1 설정: 분주비 8 (1 tick = 0.5us) TCCR1B |= (1 << CS11); } void trigger_pulse(void) { PORTE |= (1 << TRIG_PIN); _delay_us(10); PORTE &= ~(1 << TRIG_PIN); } volatile uint8_t car_detected = 0; ISR(INT5_vect) { if (echo_flag == 0) { timer_start = TCNT1; EICRB &= ~(1 << ISC50); // 하강 에지 감지로 변경 echo_flag = 1; } else if (echo_flag == 1) { timer_end = TCNT1; EICRB |= (1 << ISC50); // 다시 상승 에지 감지로 변경 echo_flag = 2; car_detected = 1; } } uint16_t measure_distance(void) { echo_flag = 0; TCNT1 = 0; trigger_pulse(); uint32_t timeout = 0; while (echo_flag < 2 && timeout < 30000) { timeout++; } if (echo_flag < 2) { return 999; // 오류 시 큰 값 반환 } uint16_t pulse = timer_end - timer_start; uint16_t distance_cm = pulse / 58; // 거리 계산 return distance_cm; } //// 스위치 입력 받기 //// int KeyMatrix() { int keyout = 0xfe; for(int i = 0; i < 4; i++) { Key_data_output = keyout; _delay_ms(1); switch (Key_data_input & 0xf0) { case 0xe0: return 0 + i; case 0xd0: return 4 + i; case 0xb0: return 8 + i; case 0x70: return 12 + i; } keyout = (keyout << 1) | 0x01; } return -1; } int cal(int k) { switch (k) { case 1: return 0; case 3: return 0xff; // enter case 4: return 1; case 5: return 2; case 6: return 3; case 7: return 15; // reset case 8: return 4; case 9: return 5; case 10: return 6; case 12: return 7; case 13: return 8; case 14: return 9; default: return -1; } }

//// 디스플레이에 출력 함수 //// void Display(int op){ uint8_t display[]= {0xc0, 0xc0, 0xc0, 0xc0}; // display 출력 값 int a0 = op % 10; // 1의 자리 int a1 = (op / 10) % 10; // 10의 자리 int a2 = (op / 100) % 10; // 100의 자리 int a3 = (op / 1000) % 10; // 1000의 자리 display[0] = numbers[a0]; display[1] = numbers[a1]; display[2] = numbers[a2]; display[3] = numbers[a3]; for(int repeat = 0; repeat <8; repeat++){ for (int i = 0; i < 4; i++) { switch (i) { case 0: PORTC = sel[0]; PORTB = display[0]; break; case 1: PORTC = sel[1]; PORTB = display[1]; break; case 2: PORTC = sel[2]; PORTB = display[2]; break; case 3: PORTC = sel[3]; PORTB = display[3]; break; } _delay_us(2000); // 깜빡임 방지를 위해 delay 늘릴 수도 있음 } } } /// UART 전송 //// void UART_init(void) { UBRR1H = 0; UBRR1L = 103; // 16MHz 기준, 9600bps (일반모드) UCSR1B = (1 << RXEN1) | (1 << TXEN1); // 송수신 활성화 UCSR1C = (1 << UCSZ11) | (1 << UCSZ10); // 8-bit 데이터 } void UART_transmit(uint8_t data) { while (!(UCSR1A & (1 << UDRE1))); // 송신 가능 대기 UDR1 = data; } uint8_t UART_receive(void) { while (!(UCSR1A & (1 << RXC1))); // 수신 완료 대기 return UDR1; } void send_int_as_uart(int num) { // 임시 배열 (최대 10자리 + NULL 문자) char buffer[10]; int i = 0; // 0일 때 예외 처리 if (num == 0) { UART_transmit('0'); return; } // 1. 숫자를 역순으로 문자로 저장 while (num > 0) { buffer[i++] = (num % 10) + '0'; // 나머지를 문자로 저장 num /= 10; } // 2. 역순이므로 거꾸로 전송 for (int j = i - 1; j >= 0; j--) { UART_transmit(buffer[j]); } UART_transmit('₩n'); // 전송 완료 표시 (선택) } // 주차 자리 상태 (1:비어있음, 0:차있음) uint8_t parking_state =

0xFF; // 8자리 모두 비어있음(1111 1111) //// led ///// void update_parking_led(void) { PORTD = parking_state; // 비어있는 자리에만 LED ON if (parking_state == 0x00) { LCD_clear(); LCD_write_string("FULL"); } } void LCD_show_parking_state(void) { LCD_goto_XY(1, 0); for (int i = 7; i >= 0; i--) { LCD_write_data((parking_state & (1 << i)) ? '1' : '0'); LCD_write_data(' '); } } /// 메인 함수 //// int main(void) { DDRC = 0xFF; // 7seg 포트 설정 DDRB = 0xff; // 7seg 포트 설정 PORTB = 0x00; PORTC = 0x00; PORTA = 0x00; //lcd 포트 DDRA = 0xff; //lcd 포트 DDRF = 0x0f; // PORT F 설정, PORT 0123 출력 4567 입력으로 사용 DDRD = 0xff; //led 출력 PORTD = 0x00; LCD_init(); UART_init(); ultrasonic_init(); sei(); // 전역 인터럽트 허용 while (1) { // 1. 입차 대기 LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Ready for entry "); LCD_goto_XY(1, 0); LCD_write_string("Please wait... "); if (UCSR1A & (1 << RXC1)) { parking_state = UART_receive(); update_parking_led(); } // 차량 감지 대기 while (1) { for (int repeat = 0; repeat < 15; repeat++) { Display(0); } if (measure_distance() < 10) break; if (parking_state == 0x00) break; } if (parking_state) continue; if (car_detected) { car_detected = 0; // LCD에 "차량번호 입력" 띄우기 LCD_clear(); // 화면 지움 // 2. 차량 감지됨 → 자리상태 표시 LCD_goto_XY(0, 0); LCD_write_string("INSERT Number"); // 문자열 출력 LCD_show_parking_state(); _delay_ms(2000); // 1초 대기 // 3. 차량번호 입력 안내 LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Input car num "); LCD_goto_XY(1, 0); LCD_write_string("on keypad... "); // 4. 차량번호 입력(4자리) + 확인('E') int car_number = 0; while (1) { int car_number = 0, digit_count = 0; while (digit_count < 4) { int key = KeyMatrix(); if (key == -1) continue; int val = cal(key); if (val == 0xff) break; // enter if (val == 15) { // reset car_number = 0; digit_count = 0; Display(0); continue; } if (val >= 0 && val <= 9) { car_number = car_number * 10 + val; digit_count++; Display(car_number); _delay_ms(200); } } } send_int_as_uart(car_number); // master로 차번호 보내기 // 6. 자리 배정 및 LED/LCD 갱신 for (int i = 7; i >= 0; i--) { if (parking_state & (1 << i)) { parking_state &= ~(1 << i); update_parking_led(); break; } } LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Parking LOT A"); LCD_show_parking_state(); _delay_ms(2000); } } }

---

Slave A code 최종본 (250701 업데이트)

#define F_CPU 16000000UL #include <avr/io.h> #include <avr/interrupt.h> #include <util/delay.h> #include <stdint.h> #include <string.h> // LCD 관련 정의 #define PORT_DATA PORTA #define PORT_CONTROL PORTG #define DDR_DATA DDRA #define DDR_CONTROL DDRG #define RS_PIN 0 #define RW_PIN 1 #define E_PIN 2 #define COMMAND_CLEAR_DISPLAY 0x01 #define COMMAND_8_BIT_MODE 0x38 // 초음파 센서 핀 정의 #define TRIG_PIN PE4 #define ECHO_PIN PE5 // 7-segment(액티브 로우) const unsigned char fnd[10] = {0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90}; // 차량번호 입력 저장 (최대 4자리) char input[5] = {0}; uint8_t input_len = 0; // 주차 자리 상태 (1:비어있음, 0:차있음) volatile uint16_t parking_state = 0xFF; // 8자리 모두 비어있음(1111 1111) // 초음파 측정용 변수 volatile uint16_t timer_start = 0; volatile uint16_t timer_end = 0; volatile uint8_t echo_flag = 0; // ---------- UART0 송신 함수 ---------- void UART0_init(void) { UBRR0H = 0; UBRR0L = 103; // 16MHz, 9600bps UCSR0B = (1 << TXEN0) | (1 << RXEN0) | (1 << RXCIE0); // 송신(TXEN0)과 수신(RXEN0) 모두 활성화 UCSR0C = (1 << UCSZ01) | (1 << UCSZ00); } void UART0_send_char(char data) { while (!(UCSR0A & (1 << UDRE0))); UDR0 = data; } void

UART0_send_string(const char *str) { while (*str) UART0_send_char(*str++); } // ---------- UART0 수신 함수 ---------- uint8_t UART0_receive(void) { while (!(UCSR0A & (1 << RXC0))); // UART0 수신 완료 대기 return UDR0; // UART0 수신 버퍼에서 데이터 반환 } //---------- UART0 interrupt 함수 ---------- volatile uint8_t master_parking_state = 0; // 마스터로부터 받은 최신 주차 상태 volatile uint8_t parking_state_updated = 0; // 새 상태가 수신되었음을 알리는 플래그 ISR(USART0_RX_vect) { char data = UDR0; if (data >= '0' && data <= '8') { // '0'에서 '8' 사이의 숫자인지 확인 (남은 자리 수) master_parking_state = data - '0'; // ASCII를 숫자로 변환 } else if (data == '₩n') { parking_state_updated = 1; } } // ---------- LCD 함수 ---------- void LCD_pulse_enable(void) { PORT_CONTROL |= (1 << E_PIN); _delay_us(1); PORT_CONTROL &= ~(1 << E_PIN); _delay_ms(1); } void LCD_write_data(uint8_t data) { PORT_CONTROL |= (1 << RS_PIN); PORT_DATA = data; LCD_pulse_enable(); _delay_ms(2); } void LCD_write_command(uint8_t command) { PORT_CONTROL &= ~(1 << RS_PIN); PORT_DATA = command; LCD_pulse_enable(); _delay_ms(2); } void LCD_clear(void) { LCD_write_command(COMMAND_CLEAR_DISPLAY); _delay_ms(2); } void LCD_init(void) { _delay_ms(50); DDR_DATA = 0xFF; PORT_DATA = 0x00; DDR_CONTROL |= (1 << RS_PIN) | (1 << RW_PIN) | (1 << E_PIN); PORT_CONTROL &= ~(1 << RW_PIN); _delay_ms(50); LCD_write_command(COMMAND_8_BIT_MODE); LCD_write_command(0x0C); // Display ON, Cursor OFF LCD_clear(); LCD_write_command(0x06); // Entry mode set } void LCD_write_string(const char *string) { uint8_t i = 0; while (string[i]) { LCD_write_data(string[i]); i++; } } void LCD_goto_XY(uint8_t row, uint8_t col) { col %= 16; row %= 2; uint8_t address = (0x40 * row) + col; uint8_t command = 0x80 + address; LCD_write_command(command); } void LCD_show_parking_state() { LCD_goto_XY(1, 0); for (int i = 7; i >= 0; i--) { if (parking_state & (1 << i)) LCD_write_data('1'); else LCD_write_data('0'); LCD_write_data(' '); } } uint8_t is_full() { return (parking_state == 0x00); } // ---------- LED 함수 ---------- void update_leds() { PORTB = ~parking_state; // 각 비트가 LED에 그대로 반영됨 (LOW일 때 켜짐) } // ---------- 초음파 함수 ---------- void ultrasonic_init(void) { DDRE |= (1 << TRIG_PIN); // TRIG 출력 DDRE &= ~(1 << ECHO_PIN); // ECHO 입력 // INT5 인터럽트 설정 (상승에지 감지) EICRB |= (1 << ISC51) | (1 << ISC50); EIMSK |= (1 << INT5); // 타이머1 설정: 분주비 8 (1 tick = 0.5us) TCCR1B |= (1 << CS11); } void trigger_pulse(void) { PORTE |= (1 << TRIG_PIN); _delay_us(10); PORTE &= ~(1 << TRIG_PIN); } ISR(INT5_vect) { if (echo_flag == 0) { timer_start = TCNT1; EICRB &= ~(1 << ISC50); // 하강 에지 감지로 변경 echo_flag = 1; } else if (echo_flag == 1) { timer_end = TCNT1; EICRB |= (1 << ISC50); // 다시 상승에지 감지로 변경 echo_flag = 2; } } uint16_t measure_distance(void) { echo_flag = 0; TCNT1 = 0; trigger_pulse(); uint32_t timeout = 0; while (echo_flag < 2 && timeout < 30000) { timeout++; } if (echo_flag < 2) { return 999; // 오류 시 큰 값 반환 } uint16_t pulse = timer_end - timer_start; uint16_t distance_cm = pulse / 58; // 거리 계산 return distance_cm; } // ---------- 7-segment 함수 ---------- void Disp_Fnd_input() { int dnum[4] = { -1, -1, -1, -1 }; for (int i = 0; i < input_len; i++) { dnum[3 - i] = input[i] - '0'; // 입력값을 오른쪽부터 채움 } for (int i = 0; i < 4; i++) { PORTC = ~(0x01 << i); // i=0: 오른쪽, i=3: 왼쪽 if (dnum[i] >= 0 && dnum[i] <= 9) PORTD = fnd[dnum[i]]; else PORTD = 0xFF; _delay_ms(2); } } // ---------- 키패드 함수 ---------- #define Key_data_input PINF #define Key_data_output PORTF

int KeyMatrix() { unsigned keyout = 0xfe; for (int i = 0; i < 4; i++) { Key_data_output = keyout; _delay_us(50); switch (Key_data_input & 0xf0) { case 0xe0: return i; // 첫 행 case 0xd0: return 4 + i; // 둘째 행 case 0xb0: return 8 + i; // 셋째 행 case 0x70: return 12 + i; // 넷째 행 } keyout = (keyout << 1) | 0x01; } return -1; } char mapKey(int key) { switch (key) { case 15: return 'D'; // 지우기 case 7: return 'E'; // 확인 case 3: return 'C'; // 미사용(X) case 12: return '7'; case 13: return '8'; case 14: return '9'; case 8: return '4'; case 9: return '5'; case 10: return '6'; case 4: return '1'; case 5: return '2'; case 6: return '3'; case 1: return '0'; default: return 0; } } // ---------- 메인 함수 ---------- int main(void) { // 7-segment, DIGIT 포트 출력 DDRD = 0xFF; PORTD = 0xFF; DDRC = 0x0F; PORTC = 0x0F; // 키패드 포트 설정 (하위 4비트 출력, 상위 4비트 입력) DDRF = 0x0F; PORTF = 0xF0; // LED 포트 설정 DDRB = 0xFF; PORTB = 0xFF; ultrasonic_init(); sei(); LCD_init(); UART0_init(); // ★ UART0 초기화 memset(input, 0, sizeof(input)); input_len = 0; char prev_key = 0; update_leds(); while (1) { // 주차 자리 업데이트 if (parking_state_updated) { parking_state_updated = 0; // 플래그 초기화 uint8_t occupied_count = 8 - master_parking_state; // 차있는 자리 수 uint16_t new_calculated_parking_state = 0xFF; // 초기화: 모두 비어있음 (1) for (int i = 0; i < occupied_count; i++) { new_calculated_parking_state &= ~(1 << (7 - i)); // 7, 6, 5... 순서로 0으로 만듦 } parking_state = new_calculated_parking_state; // 최종 비트맵으로 업데이트 update_leds(); // LED 상태 업데이트 // LCD에 업데이트된 상태 표시 (선택 사항) LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Parking update"); LCD_goto_XY(1, 0); // parking_state (uint8_t)를 문자열로 변환하여 표시 char buf[2]; itoa(master_parking_state, buf, 10); LCD_write_string("Avail: "); LCD_write_string(buf); _delay_ms(1000); // 잠시 표시 } // 만차라면 안내 메시지 및 LED 깜빡임 if (is_full()) { LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Parking Full! "); LCD_goto_XY(1, 0); LCD_write_string("No more space "); for (int repeat = 0; repeat < 15; repeat++) PORTD = 0xFF; for (int i = 0; i < 4; i++) { PORTB = 0xFF; // 모두 ON _delay_ms(200); PORTB = 0x00; // 모두 OFF _delay_ms(200); } PORTB = 0xFF; // 만차 상태에서 모두 OFF 유지 while (is_full()) { _delay_ms(500); } update_leds(); continue; if (parking_state_updated) { break; // 플래그가 1이 되면 이 루프를 빠져나가 바깥쪽 if로 이동 } _delay_ms(500); // 지연 중에도 플래그 확인 필요 } // 1. 입차 대기 LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Ready for entry "); LCD_goto_XY(1, 0); LCD_write_string("Please wait... "); while (1) { for (int repeat = 0; repeat < 15; repeat++) { Disp_Fnd_input(); } if (measure_distance() < 10) break; if (parking_state_updated) { break; // 플래그가 1이 되면 이 루프를 빠져나가 바깥쪽 if로 이동 } if (is_full()) break; } if (is_full()) continue; // 2. 차량 감지됨 → 자리상태 표시 LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Parking LOT A "); LCD_show_parking_state(); _delay_ms(2000); // 3. 차량번호 입력 안내 LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Input car num "); LCD_goto_XY(1, 0); LCD_write_string("on keypad... "); memset(input, 0, sizeof(input)); input_len = 0; // 4. 차량번호 입력(4자리) + 확인('E') while (1) { for (int repeat = 0; repeat < 15; repeat++) { Disp_Fnd_input(); } char key = 0; int k = KeyMatrix(); if (k != -1) key = mapKey(k); if (key && key != prev_key) { prev_key = key; if (key >= '0' && key <= '9') { if (input_len < 4) { input[input_len++] = key; } } else if (key == 'D') { // 지우기 if (input_len > 0) { input[--input_len] = 0; } }else if (key == 'E') { if (input_len == 4) { // 입차 처리: parking_state의 왼쪽부터 1을 0으로 for (int i = 7; i >= 0; i--) { if (parking_state & (1 << i)) { parking_state &= ~(1 << i); update_leds(); // ★ 차량

번호 4글자만 전송 for (uint8_t j = 0; j < input_len; j++) { UART0_send_char(input[j]); } // 필요하다면 개행문자 ('₩n') 추가 가능(마스터가 무시함) // UART0_send_char('₩n'); // ★★★ 반드시 아래를 추가 ★★★ memset(input, 0, sizeof(input)); input_len = 0; prev_key = 0; break; } } // 자리상태 2초간 표시 LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Parking LOT A "); LCD_show_parking_state(); _delay_ms(2000); break; } } } else if (!key) { prev_key = 0; } if (parking_state_updated) { break; // 플래그가 1이 되면 이 루프를 빠져나가 바깥쪽 if로 이동 } if (is_full()) break; } } }

DEBUGGING

7/1

출차 시 마스터에서 보낸 주차공간이 slave A에서 업데이트가 안되는 문제 발생

1. UART 간 통신 문제인지 확인 : UART0 RX ISR 함수 내부에 LED를 켜는 간단한 디버깅 코드로 통신 간 문제 없음 확인
2. Master에서 보내는 ASCII code에 해당하는 숫자가 깨져있는 지 확인 : UART0 RX ISR 로 마스터에서 신호가 가는 동시에 받은 data를 LCD에 띄우는 디버그 코드 작성. 마지막 문자열 ₩n에 해당하는 ASCII code 10 출력. 문제 없음 확인
3.

밑에 코드 돌렸을 때 10 으로 뜨는 거보면 ₩n 문자 수신 확실히 된다는 뜻이므로
UART0 interrupt에도 문제없고 마스터에서 보내는 parking state ASCII code 문자도 문제없음
ISR(USART0_RX_vect) { char data = UDR0; // --- 디버깅 코드 시작 --- LCD_clear(); LCD_goto_XY(0, 0); char buf[4]; itoa(data, buf, 10); // 정수를 문자열로 변환 LCD_write_string(buf); // 변환된 숫자 출력 _delay_ms(500); // 0.5ms 지연 (ISR은 가능한 짧아야 함) // --- 디버깅 코드 끝 --- if (data >= '0' && data <= '8') { // '0'에서 '8' 사이의 숫자인지 확인 (남은 자리 수) master_parking_state = data - '0'; // ASCII를 숫자로 변환 } else if (data == '₩n') { parking_state_updated = 1; } }

문제의 원인: main 루프의 블로킹

슬레이브 A의 main 함수는 여러 단계로 구성되어 있습니다.

- 만차 대기 루프 (while(is_full()))
- 초음파 센서 감지 대기 루프 (while(1) for distance measurement)
- 키패드 입력 대기 루프 (while(1) for keypad input)
- 그리고 각종 _delay_ms() 호출

이러한 중첩된 while 루프나 긴 지연 함수들은 main 함수의 실행 흐름을 '블로킹(blocking)' 합니다. 즉, 이 루프들이 끝나기 전까지는 main 함수의 다음 코드로 넘어가지 못합니다.
if (parking_state_updated) 블록이 가장 바깥쪽 while(1) 문 바로 밑에 하나만 있다면, 만약 main 함수가 만차 대기, 초

음파 센서 대기, 또는 키패드 입력 중인 어떤 중첩된 while 루프에 갇혀 있을 때, 마스터로부터 출차 신호가 와서 parking_state_updated 플래그가 1이 되어도 main 함수는 그 플래그를 바로 확인하지 못합니다.
main 함수는 현재 갇혀 있는 중첩 루프가 끝나고 나서야 비로소 가장 바깥쪽 while(1)의 시작 부분으로 돌아와 if (parking_state_updated) 블록을 확인할 수 있게 되는 거죠.
입차 시에는 왜 잘 되는가? 입차 시에는 차량이 진입하기 전 대기 상태이므로, 보통 main 루프가 가장 바깥쪽 while(1)의 시작 부분에 자주 도달하게 됩니다. 따라서 마스터가 입차 신호를 보내면 비교적 빠르게 parking_state_updated를 감지하고 업데이트할 수 있습니다.
출차 시에는 왜 안 되는가? 출차 시에는 차량이 나가는 과정 (특히 슬레이브 B와 마스터 간의 통신) 동안 슬레이브 A가 만차 상태이거나, 키패드 입력을 대기하거나, 다른 블로킹 루프에 들어가 있을 가능성이 높습니다. 예를 들어, 출차 처리 후 마스터가 신호를 보냈는데, 슬레이브 A가 아직 만차 루프 (while(is_full()))에 갇혀 있다면, parking_state_updated 플래그는 1이 되어도 슬레이브 A의 LCD에는 "Parking update"가 뜨지 않는 것입니다.

---

**해결책: 모든 블로킹 루프 내에서 플래그 확인**

이 문제를 해결하려면, main 함수 내의 모든 블로킹(blocking)될 수 있는 중첩된 while 루프 안에 parking_state_updated 플래그를 확인하고, 플래그가 1이면 해당 루프를 break하여 가장 바깥쪽 while(1)의 업데이트 블록으로 빠져나오도록 해야 합니다.
이 방법은 제가 이전 답변에서 여러 번 강조했던 내용이기도 합니다. 이 로직을 모든 중첩 루프에 빠짐없이 적용해야 합니다.
int main(void) { // ... (기존 초기화 코드) ... while (1) { // --- `parking_state` 업데이트 로직 (가장 바깥쪽에 하나만 둠) --- if (parking_state_updated) { parking_state_updated = 0; // 플래그 초기화 // ... (master_parking_state를 parking_state 비트맵으로 변환) ... // ... (update_leds(); 및 LCD 표시 로직) ... // (이곳에 PB4 LED 켜고 끄는 디버깅 코드를 넣어보면, // 출차 시 여기가 실행되는지 확인할 수 있습니다.) } // ---------- // 1. 만차 대기 루프 while (is_full()) { // ... 만차 안내 코드 ... // ★ 중요: 마스터 업데이트 감지 시 루프 탈출 if (parking_state_updated) { break; // 플래그가 1이 되면 이 루프를 빠져나가 바깥쪽 if로 이동 } _delay_ms(500); // 지연 중에도 플래그 확인 필요 } // 만약 만차 루프에서 나왔다면 (차량 빠짐) 다시 바깥쪽 while(1) 시작으로 돌아가 // `if (parking_state_updated)`를 다시 확인하도록 `continue;`를 고려할 수 있습니다. // if (parking_state_updated) continue; // 플래그가 설정되어있으면 즉시 업데이트 로직으로 // 2. 초음파 센서 감지 대기 루프 while (1) { // 초음파 센서 대기 for (int repeat = 0; repeat < 15; repeat++) { Disp_Fnd_input(); // FND 업데이트 중 } if (measure_distance() < 10) break; // 차량 감지 시 탈출 // ★ 중요: 대기 중 마스터 업데이트 감지 시 루프 탈출 if (parking_state_updated) { break; // 플래그가 1이 되면 이 루프를 빠져나가 바깥쪽 if로 이동 } if (is_full()) break; // 갑자기 만차가 되면 탈출 (비상 조건) } // 위 루프에서 is_full()로 인해 빠져나왔다면 다시 메인 while 시작으로 if (is_full()) continue; // 3. 차량번호 입력 대기 루프 while (1) { // 키패드 입력 대기 for (int repeat = 0; repeat < 15; repeat++) { Disp_Fnd_input(); // FND 업데이트 중 } // ... 키패드 입력 처리 로직 ... // ★ 중요: 키패드 입력 대기 중 마스터 업데이트 감지 시 루프 탈출 if (parking_state_updated) { break; // 플래그가 1이 되면 이 루프를 빠져나가 바깥쪽 if로 이동 } if (is_full()) break; // 키패드 입력 중 만차가 되면 탈출 } // ... (나머지 코드) ... } }

**마지막 확인**

1. 모든 중첩 while 루프: main 함수 내에 있는 모든 while 루프 (특히 조건이 항상 true이거나 특정 이벤트 대기인 루프)를 찾아 위와 같이 if (parking_state_updated) break; 로직을 추가해야 합니다.
2. _delay_ms() 내부에서의 인터럽트: _delay_ms() 함수 자체는 인터럽트를 막지 않으므로, 이 함수가 실행되는 동안에도 UART 인터럽트는 발생하고 parking_state_updated 플래그는 설정됩니다. 문제는 _delay_ms()가 끝난 후, 그리고 현재 while 루프가 끝난 후에야 if (parking_state_updated)가 확인된다는 점입니다.

이러한 블로킹 루프를 수정하면, 마스터가 출차 신호를 보내서 parking_state_updated 플래그가 설정되었을 때, 슬레이브 A가 더 이상 다른 작업에 갇히지 않고 빠르게 플래그를 감지하여 "Parking update" 메시지를 띄우고 parking_state를 갱신할 수 있을 것입니다.
이 부분이 현재 문제의 가장 유력한 원인입니다. 모든 중첩된 while 루프를 확인하여 if (parking_state_updated) { break; } 코드를 추가하고 다시 테스트해보세요.

**만차 FULL 일 때 다시 멈춤 현상 ?**

// 만차라면 안내 메시지 및 LED 깜빡임 if (is_full()) { LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Parking Full! "); LCD_goto_XY(1, 0); LCD_write_string("No more space "); for (int repeat = 0; repeat < 15; repeat++) PORTD = 0xFF; // FND 끄는 동작 for (int i = 0; i < 4; i++) { // LED 깜빡임 PORTB = 0xFF; _delay_ms(200); PORTB = 0x00; _delay_ms(200); } PORTB = 0xFF; // 만차 상태에서 모두 OFF 유지 // ★★★ 여기가 문제 ★★★ while (is_full()) { // 만차 상태가 풀릴 때까지 대기 // ★ 마스터 업데이트 감지 시 루프 탈출 if (parking_state_updated) { // 플래그가 1이 되면 이 `while(is_full())` 루프를 빠져나감 break; } _delay_ms(500); // 주기적으로 상태 확인 } // `while(is_full())` 루프를 빠져나오면 가장 바깥 `while(1)`의 시작 부분으로 돌아가서 `parking_state_updated`를 확인하게 됨. // 따라서 `update_leds()`와 `continue;`는 여기서는 필요 없음. // `continue;`를 제거해야만 만차가 풀렸을 때 다른 루틴으로 진행됩니다. // `update_leds()`는 바깥 `if (parking_state_updated)` 블록이나, 만차 해제 후의 초기화 단계에서 호출되는 것이 좋습니다. (위에서 이미 호출)

# atmega128 : 스마트 주차 시스템 ; Master system

project

2025/06/16 20:45 http://blog.naver.com/1225seong/223901371171

/* * Master.c * 입차 번호 누르고 확인 누른 순간 -> 입차로 감지 * LCD에 입차 감지 표시. 아닐 때는 기다리고 있다고 LCD에 계속 표시 * UART 통해서 입차 대수 (최대 8대) 남은 공간 계속 신호로 slave A에 알려주기 * 차가 들어오는 순간부터 타이머 인터럽트로 1초씩 시간 계산 -> 메모리에 차번호 4자리, 시간 저장하기 * slave B에서 출차 신호를 보내서 master에서 신호가 수신되면 주차 시간만큼 요금 정산하기 * 입차 흐름: 1. 대기 상태 → LCD: "입차 대기 중입니다" 2. slave A에서 차량 번호 입력 (Keypad) 확인 누름 (#) → 입차 감지 3. UART 인터럽트 들어옴 - LCD: "입차 감지됨" - 차량번호 + 시간 메모리에 저장 - car_count++, UART로 Slave A에 잔여 공간 전송 - 입차 시간 저장 출차 흐름: - main 루프 내에서 UART로 slave B가 차량번호 송신 → 출차 요청 - 그 번호와 메모리에 저장된 번호 비교 → 있으면 시간 계산 - 요금 정산, LCD 출력 and UART 송신 - car_count--, 잔여 공간 다시 Slave A로 송신 * Created: 2025-06-16 오전 9:27:58 * Author : me */

#define F_CPU 16000000UL #include <avr/io.h> #include <avr/interrupt.h> #include <util/delay.h> #include <string.h> // strcpy 함수 쓰려면 포함 #include <stdlib.h> // itoa() 함수 쓰려면 포함 // LCD 관련 정의 #define PORT_DATA PORTA #define PORT_CONTROL PORTG #define DDR_DATA DDRA #define DDR_CONTROL DDRG #define RS_PIN 0 #define RW_PIN 1 #define E_PIN 2 #define COMMAND_CLEAR_DISPLAY 0x01 #define COMMAND_8_BIT_MODE 0x38 #define MAX_CARS 8 // 차량 번호, 시간, 빈자리 관리 typedef struct { char car_num[5]; // 차량번호 (NULL 포함해서 5바이트 확보) uint16_t entry_seconds; // 입차 시간 uint8_t valid; // 1: 주차 중, 0: 빈 자리 } CarData; CarData parked_cars[MAX_CARS]; // 차량 정보 메모리에 저장 // 주차 자리 상태 uint8_t calculate_parking_state(void) { uint8_t occupied = 0; for (int i = 0; i < MAX_CARS; i++) { if (parked_cars[i].valid == 1) { occupied++; } } return MAX_CARS - occupied; // 남은 공간 수 반환 } // ---------- LCD 함수 ---------- void LCD_pulse_enable(void) { PORT_CONTROL |= (1 << E_PIN); _delay_us(1); PORT_CONTROL &= ~(1 << E_PIN); _delay_ms(1); } void LCD_write_data(uint8_t data) { PORT_CONTROL |= (1 << RS_PIN); PORT_DATA = data; LCD_pulse_enable(); _delay_ms(2); } void LCD_write_command(uint8_t command) { PORT_CONTROL &= ~(1 << RS_PIN); PORT_DATA = command; LCD_pulse_enable(); _delay_ms(2); } void LCD_clear(void) { LCD_write_command(COMMAND_CLEAR_DISPLAY); _delay_ms(2); } void LCD_init(void) { _delay_ms(50); DDR_DATA = 0xFF; PORT_DATA = 0x00; DDR_CONTROL |= (1 << RS_PIN) | (1 << RW_PIN) | (1 << E_PIN); PORT_CONTROL &= ~(1 << RW_PIN); _delay_ms(50); LCD_write_command(COMMAND_8_BIT_MODE); LCD_write_command(0x0C); // Display ON, Cursor OFF LCD_clear(); LCD_write_command(0x06); // Entry mode set } void LCD_write_string(const char *string) { uint8_t i = 0; while (string[i]) { LCD_write_data(string[i]); i++; } } void LCD_goto_XY(uint8_t row, uint8_t col) { col %= 16; row %= 2;

uint8_t address = (0x40 * row) + col; uint8_t command = 0x80 + address; LCD_write_command(command); } // ---------- UART 함수 ---------- void UART0_init(void) { UBRR0H = 0; UBRR0L = 103; UCSR0B = (1 << RXEN0) | (1 << TXEN0) | (1 << RXCIE0); // 수신 인터럽트 포함 UCSR0C = (1 << UCSZ01) | (1 << UCSZ00); } void UART1_init(void) { UBRR1H = 0; UBRR1L = 103; // 9600 baud @16MHz UCSR1B = (1<<RXEN1)|(1<<TXEN1)|(1<<RXCIE1); UCSR1C = (1<<UCSZ11)|(1<<UCSZ10); } // 입차 UART 0로 송신 void UART0_transmit(uint8_t data) { while (!(UCSR0A & (1 << UDRE0))); UDR0 = data; } void UART_transmit_string(const char* str) { while (*str) { UART0_transmit(*str++); } } // ---------- UART 인터럽트 함수 ---------- // 입차 신호 발생 시 동작 volatile char rx_buffer[5]; // 차량번호 수신 버퍼 volatile uint8_t rx_index = 0; volatile uint8_t new_car_ready = 0; ISR(USART0_RX_vect) { char data = UDR0; if (rx_index < 4) { rx_buffer[rx_index++] = data; } if (rx_index == 4) { rx_buffer[4] = '₩0'; // 문자열 종료 new_car_ready = 1; rx_index = 0; } } // 출차 신호 발생 시 동작 volatile char exit_car_buf[7]; volatile uint8_t exit_index = 0; volatile uint8_t exit_ready = 0; volatile uint8_t exit_done = 0; ISR(USART1_RX_vect) { char data = UDR1; if (data == '₩n') { if (exit_index < sizeof(exit_car_buf)) exit_car_buf[exit_index] = '₩0'; // 문자열 종료 else exit_car_buf[sizeof(exit_car_buf)-1] = '₩0'; exit_ready = 1; exit_index = 0; // 디버깅용 LCD 출력 LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("RCV:"); LCD_write_string(exit_car_buf); } else { if (exit_index < sizeof(exit_car_buf) - 1) { exit_car_buf[exit_index++] = data; } else { exit_index = 0; // overflow 방지 } } if (data == 'D') { // 'D' 문자가 오면 exit_done 플래그 세우기 exit_done = 1; } } // ---------- 타이머 인터럽트 함수 ---------- volatile uint16_t current_seconds = 0; // 전역 시간 volatile uint8_t count = 0; ISR(TIMER0_COMP_vect) { count++; if (count == 61) { count = 0; current_seconds++; } } void Compare_INT(void){ TCCR0 |= (1 << CS02) | (1 << CS01) | (1 << CS00); // 분주비 1024로 설정 (1 1 1) TIMSK |= (1 << OCIE0); // 비교일치 interrupt 허용 OCR0 = 255; // 비교일치 기준 값. 65ns * 1024 = 66us >> 256번 했을 때 비교기 1번 카운트 됨 // 16MHz => T = 65ns, 65ns * 1024 (분주비) = 66560ns = 66us * 256 (OCR 값) = 8448us * 64 *2 (비교일치 횟수)= 541ms*2 = 1s sei(); }
uint8_t parking_state =0; uint8_t waiting_displayed = 0; int main(void) { UART0_init(); UART1_init(); Compare_INT(); LCD_init(); // UART 핀 설정 DDRE |= (1 << PE1); // TX0 DDRE &= ~(1 << PE0); // RX0 DDRD |= (1 << PD1); // TX1 DDRD &= ~(1 << PD0); // RX1 // 차량 데이터 초기화 for (int i = 0; i < MAX_CARS; i++) { parked_cars[i].valid = 0; memset(parked_cars[i].car_num, 0, 5); } parking_state = calculate_parking_state(); while (1) { // 1. 입차 처리 if (new_car_ready) { new_car_ready = 0; int idx = -1; for (int i = 0; i < MAX_CARS; i++) { if (parked_cars[i].valid == 0) { idx = i; break; } } if (idx != -1) { strncpy(parked_cars[idx].car_num, rx_buffer, 4); parked_cars[idx].car_num[4] = '₩0'; parked_cars[idx].entry_seconds = current_seconds; parked_cars[idx].valid = 1; LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("CAR DETECTED"); LCD_goto_XY(1, 0); LCD_write_string(parked_cars[idx].car_num); parking_state = calculate_parking_state(); UART0_transmit('0' + parking_state); UART0_transmit('₩n'); _delay_ms(1000); waiting_displayed = 0; } else { // 만차 parking_state = 0; UART0_transmit('0' + parking_state); UART0_transmit('₩n'); LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("PARKING FULL"); _delay_ms(1000); waiting_displayed = 0; } } // 2. 대

기 표시 else if (!new_car_ready && !waiting_displayed) { parking_state = calculate_parking_state(); char buff[16]; itoa(parking_state, buff, 10); LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("CAR WAITING"); LCD_goto_XY(1, 0); LCD_write_string("Available: "); LCD_write_string(buff); waiting_displayed = 1; } // 3. 출차 처리 else if (exit_ready) { exit_ready = 0; uint8_t found = 0; // 차량번호 추출 (앞 4자리) char exit_num_only[5]; strncpy(exit_num_only, exit_car_buf, 4); exit_num_only[4] = '₩0'; // 디버깅용 출력 LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Exit Num:"); LCD_write_string(exit_num_only); _delay_ms(500); for (int i = 0; i < MAX_CARS; i++) { if (parked_cars[i].valid && strncmp(exit_num_only, parked_cars[i].car_num, 4) == 0) { found = 1; // 디버깅용 출력 LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Match Found:"); LCD_write_string(parked_cars[i].car_num); _delay_ms(500); uint16_t duration = current_seconds - parked_cars[i].entry_seconds; uint16_t fee = duration * 10; char buf[16]; LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("EXIT:"); LCD_write_string(exit_num_only); itoa(fee, buf, 10); LCD_goto_XY(1, 0); LCD_write_string("FEE:"); LCD_write_string(buf); LCD_write_string(" Won"); // 슬레이브 B로 duration, fee 전송 char txbuf[20]; itoa(duration, txbuf, 10); uint8_t len = strlen(txbuf); txbuf[len] = ','; itoa(fee, txbuf + len + 1, 10); len = strlen(txbuf); txbuf[len] = '₩n'; txbuf[len + 1] = '₩0'; for (uint8_t k = 0; txbuf[k] != 0; k++) { while (!(UCSR1A & (1 << UDRE1))); UDR1 = txbuf[k]; } // 슬레이브 B에서 'D' 받을 때까지 대기 uint32_t timeout = 0; while (!exit_done && timeout++ < 500000UL) { _delay_us(10); } if (exit_done) { exit_done = 0; parked_cars[i].valid = 0; memset(parked_cars[i].car_num, 0, 5); parked_cars[i].entry_seconds = 0; parking_state = calculate_parking_state(); UART0_transmit('0' + parking_state); UART0_transmit('₩n'); LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("EXIT DONE"); exit_ready = 0; exit_done = 0; exit_index = 0; memset(exit_car_buf, 0, sizeof(exit_car_buf)); _delay_ms(1000); waiting_displayed = 0; }else { // 타임아웃 발생했을 때 디버깅 출력 LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("EXIT TIMEOUT"); _delay_ms(1000); } break; } } if (!found) { LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("EXIT FAILED"); LCD_goto_XY(1, 0); LCD_write_string("NOT FOUND"); _delay_ms(1000); waiting_displayed = 0; } } } }

**프로토콜 추가하기**

**커맨드 종류**

| 커맨드 | 방향 | 데이터 형식 | 설명 |
|---|---|---|---|
| 0x45 | Slave→Master | CNNN | 차량번호 전송 (예: C1234) |
| 0x41 | Master→Slave | SX | 주차 가능 대수 (X: 0-8) |
| 0x4E | 양방향 | - | 수신 확인(ACK) |
| 0x52 | 양방향 | - | 재전송 요청(NAK) |

| 코드 | 의미 | 방향 |
|---|---|---|

| | | |
|---|---|---|
| Ixxxx₩n | 입차 요청 | Slave A → Master |
| Sx₩n | 남은 자리 | Master → Slave A |
| Oxxxx₩n | 출차 요청 | Slave B → Master |
| Fxxxx yyyy₩n | 요금 응답 | Master → Slave B |

에러검출 추가하기

# 6/23 스마트 주차 시스템: Slave B 출차 시스템 및 진행상황

project

2025/06/23 19:15 http://blog.naver.com/1225seong/223909039532

Slave B code

// 출차 코드 #define F_CPU 16000000UL #include <avr/io.h> #include <avr/interrupt.h> #include <util/delay.h> #include <stdint.h> #include <string.h> #include <stdlib.h> // LCD 관련 정의 #define PORT_DATA PORTA #define PORT_CONTROL PORTG #define DDR_DATA DDRA #define DDR_CONTROL DDRG #define RS_PIN 0 #define RW_PIN 1 #define E_PIN 2 #define TRIG_PIN PE4 #define ECHO_PIN PE5 // ---- 세그먼트 배열과 로직은 기존 그대로 ---- const unsigned char fnd[10] = {0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90}; char input[5] = {0}; uint8_t input_len = 0; volatile uint16_t parking_minutes = 0; volatile uint16_t parking_fee = 0; volatile uint8_t payment_ready = 0; volatile uint16_t timer_start = 0; volatile uint16_t timer_end = 0; volatile uint8_t echo_flag = 0; // UART1 초기화 void UART1_init(void) { UBRR1H = 0; UBRR1L = 103; // 9600bps 기준 (F_CPU = 16MHz) UCSR1B = (1 << TXEN1) | (1 << RXEN1) | (1 << RXCIE1); // 송신, 수신, 인터럽트 활성화 UCSR1C = (1 << UCSZ11) | (1 << UCSZ10); // 8-bit data } // UART1 단일 문자 송신 void UART1_send_char(char data) { while (!(UCSR1A & (1 << UDRE1))); // 송신 버퍼가 빌 때까지 대기 UDR1 = data; } // UART1 문자열 송신 void UART1_send_string(const char *str) { while (*str) UART1_send_char(*str++); } // LCD void LCD_pulse_enable(void) { PORT_CONTROL |= (1 << E_PIN); _delay_us(1); PORT_CONTROL &= ~(1 << E_PIN); _delay_ms(1); } void LCD_write_data(uint8_t data) { PORT_CONTROL |= (1 << RS_PIN); PORT_DATA = data; LCD_pulse_enable(); _delay_ms(2); } void LCD_write_command(uint8_t command) { PORT_CONTROL &= ~(1 << RS_PIN); PORT_DATA = command; LCD_pulse_enable(); _delay_ms(2); } void LCD_clear(void) { LCD_write_command(0x01); _delay_ms(2); } void LCD_init(void) { _delay_ms(50); DDR_DATA = 0xFF; PORT_DATA = 0x00; DDR_CONTROL |= (1 << RS_PIN) | (1 << RW_PIN) | (1 << E_PIN); PORT_CONTROL &= ~(1 << RW_PIN); _delay_ms(50); LCD_write_command(0x38); LCD_write_command(0x0C); LCD_clear(); LCD_write_command(0x06); } void LCD_write_string(const char *string) { uint8_t i = 0; while (string[i]) { LCD_write_data(string[i]); i++; } } void LCD_goto_XY(uint8_t row, uint8_t col) { col %= 16; row %= 2; uint8_t address = (0x40 * row) + col; uint8_t command = 0x80 + address; LCD_write_command(command); } // 초음파 void ultrasonic_init(void) { DDRE |= (1 << TRIG_PIN); DDRE &= ~(1 << ECHO_PIN); PORTE |= (1 << ECHO_PIN); TCCR1B |= (1 << CS11); } void trigger_pulse(void) { PORTE |= (1 << TRIG_PIN); _delay_us(10); PORTE &= ~(1 << TRIG_PIN); } uint16_t measure_distance(void) { echo_flag = 0; TCNT1 = 0; trigger_pulse(); uint32_t timeout = 0; while (!(PINE & (1 << ECHO_PIN))) { if (timeout++ > 48000) return 999; } uint16_t start = TCNT1; timeout = 0; while (PINE & (1 << ECHO_PIN)) { if (timeout++ > 48000) return 999; } uint16_t end = TCNT1; uint16_t pulse; if (end >= start)

pulse = end - start; else pulse = (65535 - start) + end; return (pulse * 0.5) / 58; } // ---- 세그먼트 표시 함수: PORTD → PORTB, PORTC → PORTF ---- void Disp_Fnd_input() { for (int i = 0; i < 4; i++) { PORTF = 0xFF; // 모든 자릿수 비활성화 _delay_us(10); if (i < input_len && input[i] >= '0' && input[i] <= '9') PORTB = fnd[input[i] - '0']; else PORTB = 0xFF; PORTF = ~(0x08 >> i); // 해당 자릿수 활성화 _delay_ms(2); } } // ---- 키패드 함수: PORTF/PINF → PORTC/PINC ---- #define Key_data_input PINC #define Key_data_output PORTC int KeyMatrix() { PORTF = 0xFF; // 세그먼트 자릿수 비활성화 unsigned keyout = 0xfe; for (int i = 0; i < 4; i++) { Key_data_output = keyout; _delay_us(50); switch (Key_data_input & 0xf0) { case 0xe0: return i; case 0xd0: return 4 + i; case 0xb0: return 8 + i; case 0x70: return 12 + i; } keyout = (keyout << 1) | 0x01; } return -1; } char mapKey(int key) { switch (key) { case 15: return 'D'; case 7: return 'E'; case 3: return 'C'; case 12: return '7'; case 13: return '8'; case 14: return '9'; case 8: return '4'; case 9: return '5'; case 10: return '6'; case 4: return '1'; case 5: return '2'; case 6: return '3'; case 1: return '0'; default: return 0; } } // UART1 수신 인터럽트 ISR(USART1_RX_vect) { static char buf[16]; static uint8_t idx = 0; char data = UDR1; if (data == '₩n') { buf[idx] = 0; char *token = strtok(buf, ","); if (token) parking_minutes = atoi(token); token = strtok(NULL, ","); if (token) parking_fee = atoi(token); payment_ready = 1; idx = 0; } else { if (idx < 15) buf[idx++] = data; } } // 정수→문자열 변환 함수 void int_to_str(uint16_t num, char *buf) { char tmp[6]; int i = 0, j = 0; if(num == 0) { buf[0] = '0'; buf[1] = 0; return; } while(num > 0) { tmp[i++] = (num % 10) + '0'; num /= 10; } for(j = 0; j < i; j++) buf[j] = tmp[i-j-1]; buf[j] = 0; }

int main(void) { // ---- 포트 설정: 요청한 부분만 변경 ---- DDRB = 0xFF; // 7-segment 데이터 (PORTB) PORTB = 0xFF; DDRF = 0x0F; // 7-segment 자릿수 (PORTF) PORTF = 0xFF; DDRC = 0x0F; // 키패드 (PORTC) PORTC = 0xF0; // 키패드 상위 4비트 풀업 DDR_DATA = 0xFF; DDR_CONTROL |= (1 << RS_PIN) | (1 << RW_PIN) | (1 << E_PIN); LCD_init(); ultrasonic_init(); UART1_init(); sei(); memset(input, 0, sizeof(input)); input_len = 0; char prev_key = 0; while (1) { LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Exit System "); LCD_goto_XY(1, 0); LCD_write_string("Ready... "); _delay_ms(500); while (1) { for (int repeat = 0; repeat < 15; repeat++) { Disp_Fnd_input(); } if (measure_distance() < 10) break; } LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Input car num "); LCD_goto_XY(1, 0); LCD_write_string("on Keypad... "); memset(input, 0, sizeof(input)); input_len = 0; while (1) { for (int repeat = 0; repeat < 15; repeat++) { Disp_Fnd_input(); } char key = 0; int k = KeyMatrix(); if (k != -1) key = mapKey(k); if (key && key != prev_key) { prev_key = key; if (key >= '0' && key <= '9') { if (input_len < 4) { input[input_len++] = key; } } else if (key == 'D') { if (input_len > 0) { input[--input_len] = 0; } } else if (key == 'E') { if (input_len == 4) { for (uint8_t j = 0; j < input_len; j++) { UART1_send_char(input[j]); } UART1_send_char('₩n'); LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Waiting master.."); LCD_goto_XY(1, 0); LCD_write_string("for fee info "); uint16_t wait = 0; while (!payment_ready && wait++ < 2000) { _delay_ms(1); } LCD_clear(); if (payment_ready) { char buf1[8], buf2[8]; LCD_goto_XY(0, 0); LCD_write_string("Time: "); int_to_str(parking_minutes, buf1); LCD_write_string(buf1); LCD_write_string(" min"); LCD_goto_XY(1, 0); LCD_write_string("Fee: "); int_to_str(parking_fee, buf2); LCD_write_string(buf2); LCD_write_string(" won"); LCD_goto_XY(1, 13);

LCD_write_string("Pay"); while (1) { int k2 = KeyMatrix(); char key2 = 0; if (k2 != -1) key2 = mapKey(k2); if (key2 == 'E') { // 여기에 차량번호 + 결제 완료 신호 전송! for (uint8_t j = 0; j < input_len; j++) { UART1_send_char(input[j]); } UART1_send_char(','); // 구분자 UART1_send_char('D'); // Done 신호 UART1_send_char('₩n'); // 끝 break; } _delay_ms(50); } LCD_clear(); LCD_goto_XY(0, 0); LCD_write_string("Payment done "); LCD_goto_XY(1, 0); LCD_write_string("Thanks! "); _delay_ms(2000); payment_ready = 0; } else { LCD_goto_XY(0, 0); LCD_write_string("No master resp. "); LCD_goto_XY(1, 0); LCD_write_string("Try again later "); _delay_ms(2000); } memset(input, 0, sizeof(input)); input_len = 0; prev_key = 0; break; } } } else if (!key) { prev_key = 0; } } } }

---

진행상황

진행상황 —---------- 0622 입차 -> 출차 요금 계산 정산 가능 근데 딱 한 대만. 안되는 점 : 마스터 MCU에서 입차로 빈자리 정보 보내기 마스터에서 입차로 TX 꽂으면 안 굴러감 출차에서 요금 정산 완료가 떠야 마스터에 빈자리 표시가 돼야 되는데 요금 알림만 떠도 빈자리로 떠버리는거 더 추가할거 회차 모터로 차단기 구현 입차 차량번호가 이미 지금 주차장에 들어와있는 차량 같은 번호면 입차 불가라고 뜨게 하기 —---------- 주차 시스템 현황 분석 및 개선안 현재 구현된 기능 입차 → 출차 → 요금 정산 기본 플로우 완료 단일 차량에 대한 주차 시간/요금 계산 정상 작동 LCD 인터페이스를 통한 사용자 알림 주요 문제점 graph TD A[입차 시스템] -->|TX 연결| B[마스터 MCU] B -->|빈자리 정보 전송 실패| A C[출차 시스템] -->|요금 계산| D[마스터 MCU] D -->|미결제 상태에서 빈자리 표시| E[데이터 불일치] 해결 방안 UART 통신 안정화 하드웨어: TX/RX 핀 풀업 저항 추가 소프트웨어: 패킷 구조 재설계 c // 기존: '3' -> 빈자리 수 // 개선: "$FREE,3₩n" (프로토콜 명시) 출차 상태 머신 강화 // 기존 코드 if (payment_ready) { /* 요금 표시 */ } // 개선 코드 enum {FEE_DISPLAY, PAYMENT_WAIT, COMPLETED} exit_state; if (exit_state == COMPLETED) update_slot_count(); 중복 차량 번호 검증 int is_duplicate(char* new_num) { for (int i=0; i<MAX_CARS; i++) { if (parked_cars[i].valid && strncmp(new_num, parked_cars[i].car_num, 4)==0) return 1; } return 0; } 시스템 개선 제안서 1. 통신 프로토콜 표준화 패킷 유형 송신자 수신자 형식 설명 SlotUpdate 마스터 입차 "$FREE,N₩n" 빈 자리 수 (N=0~8) EntryRequest 입차 마스터 "$ENT,ABCD₩n" 차량번호 ABCD 입차 요청 ExitData 마스터 출차 "$TIME,MM₩n" 주차 시간(분) Payment 출차 마스터 "$PAID,ABCD₩n" 차량번호 ABCD 결제 완료 2. 차단기 제어 시스템 graph LR A[초음파 센서] -->|차량 감지| B[아두이노] B -->|PWM 신호| C[서보 모터] C --> D[차단기 제어] D -->|개방| E[차량 통과] E -->|통과 완료| F[차단기 폐쇄] 핀 구성 #define ENTRY_SERVO_PIN PB1 // 입차 차단기 #define EXIT_SERVO_PIN PB2 // 출차 차단기 void open_barrier(uint8_t pin) { for (int pos=0; pos<=90; pos++) { analogWrite(pin, pos); delay(15); } } 3. 중복 차량 방지 로직 // 입차 처리 시 추가 if (is_duplicate(rx_buffer)) { LCD_clear(); LCD_write_string("ALREADY PARKED"); UART_transmit_string("$DUP₩n"); // 중복 알림 return; } 4. 상태 관리 강화 주차 상태 다이어그램 stateDiagram-v2 [*] --> Idle Idle --> CarDetected : 차량 감지 CarDetected --> CheckSlot : 빈자리 확인 CheckSlot --> Reject : 자리 없음 CheckSlot --> AssignSlot : 자리 있음 AssignSlot --> OpenBarrier : 차단기 개방 OpenBarrier --> [*] : 차량 입장 완료 Idle --> ExitRequest : 출차 요청 ExitRequest --> FeeCalculation : 요금 계산 FeeCalculation --> PaymentWait

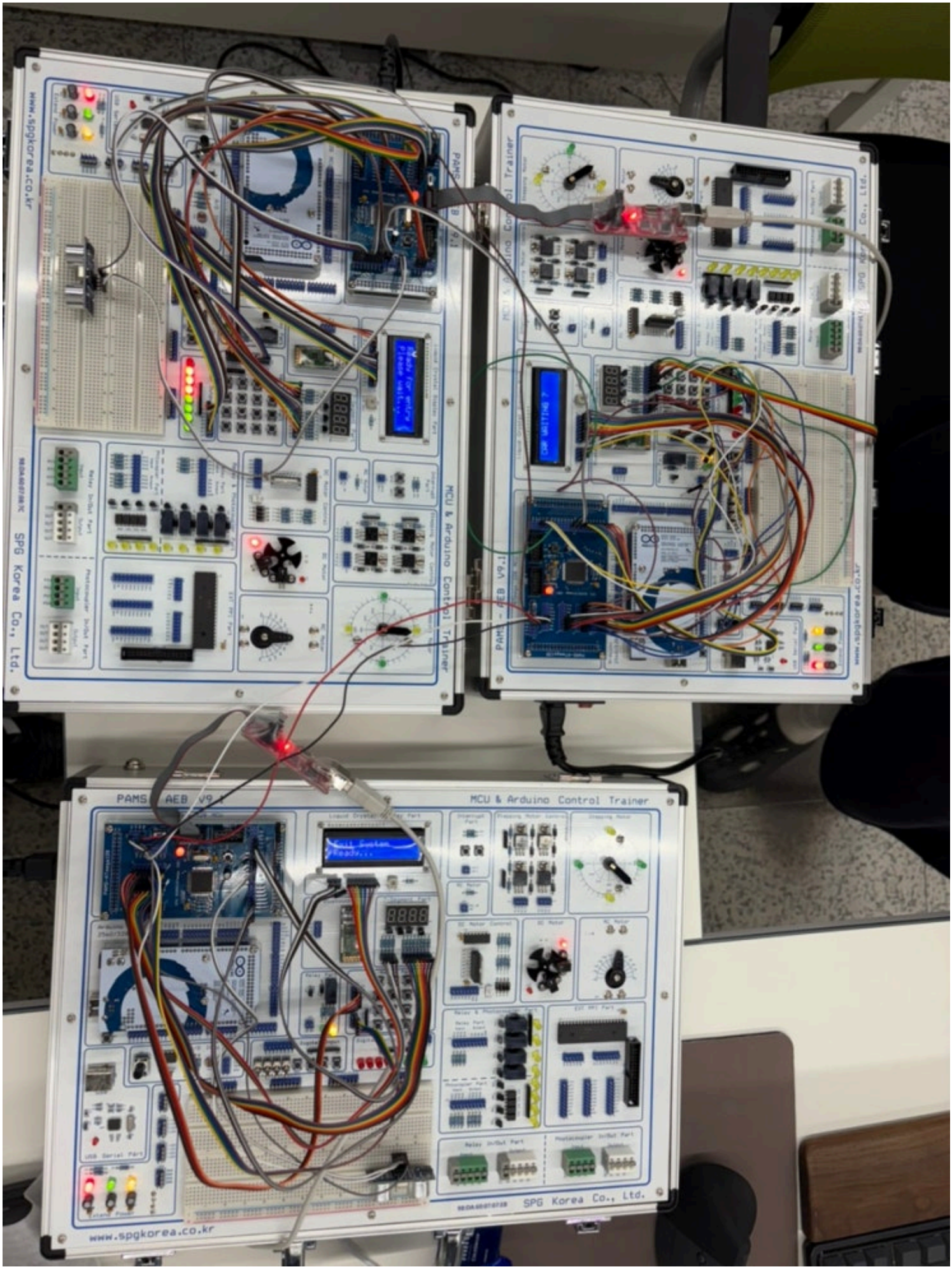
: 결제 대기 PaymentWait --> Complete : 결제 완료 Complete --> OpenBarrier : 차단기 개방 OpenBarrier --> [*] : 차량 퇴장 완료 최종 시스템 아키텍처 [입차 시스템] ← UART → [마스터 MCU] ← UART → [출차 시스템] │ │ │ ├── 초음파 센서 ├── LCD 상태 표시 ├── 키패드 ├── 키패드 ├── EEPROM 저장 ├── 7-Segment └── 서보 모터 └── RTC 모듈 └── 서보 모터 개선 포인트 시간 동기화: RTC 모듈 추가 #include <RTClib.h> DateTime now = rtc.now(); parked_cars[idx].entry_time = now.unixtime(); 데이터 영속성: EEPROM에 주차 정보 저장 #include <EEPROM.h> EEPROM.put(0, parked_cars); 에러 핸들링: 명시적 오류 코드 enum { ERR_OK = 0, ERR_DUPLICATE = 0x81, ERR_SLOT_FULL = 0x82, ERR_INVALID_EXIT = 0x83 }; —----------

입차 -> 마스터 -> 출차 -> 마스터 까진 완료

# ATmega128 프로젝트: 스마트 주차 시스템 완성본

project

2025/07/01 21:24 http://blog.naver.com/1225seong/223918408799


www.spgkorea.co.kr
SPG Korea Co., Ltd.
MCU & Arduino Control Trainer
PAMS - AEB V9.1
MCU & Arduino Control Trainer
SPG Korea Co., Ltd.
PAMS - AEB V9.1
MCU & Arduino Control Trainer
SPG Korea Co., Ltd.
www.spgkorea.co.kr

회로도 구성 ( 위 : master, 아래 : slave A, 오른옆 : slave B)

**추가하면 좋을 로직**

· 에러체크

· 모터 차단기 구현

· 중복 차량 체크

· 정기 주차 차량 요금 할인 / 특정 차량 요금 면제

정기 주차 차량 요금 할인 / 특정 차량 요금 면제

**슬레이브 A에서 중복 차량 검증을 하려면?**

슬레이브 A에서 중복 차량 검증을 하려면 몇 가지 방법이 있습니다.

**▶ 마스터에게 중복 여부를 물어보는 방식 (권장)**

가장 현실적이고 효율적인 방법입니다. 슬레이브 A는 마스터에게 차량 번호를 보내면서 "이 번호 중복이니?"라고 물어보고, 마스터가 "응, 중복이야" 또는 "아니, 중복 아니야"라고 응답해주는 방식입니다.

**프로토콜 변경 (마스터와 슬레이브 A 간):**

- **슬레이브 A -> 마스터 (차량 번호 중복 조회 요청):**
- 메시지 형식: [4자리 차량 번호],C₩n (여기서 C는 'Check Duplicate'의 약자)
- 예시: 1234,C₩n
- **마스터 -> 슬레이브 A (중복 여부 응답):**
- 메시지 형식: OK₩n (중복 아님) 또는 DUP₩n (중복임)
- 예시: OK₩n 또는 DUP₩n

**슬레이브 A 코드에 추가될 로직:**

1. 사용자가 4자리 차량 번호를 입력하고 'E'를 누르면, UART0_send_string("1234,C₩n"); 와 같이 마스터에게 전송합니다.
2. 마스터의 응답을 기다립니다 (OK₩n 또는 DUP₩n). 이때 payment_ready 플래그와 유사한 duplicate_check_ready 플래그를 사용하여 마스터 응답을 기다립니다.
3. 응답이 DUP₩n이면 "Duplicate car!" 메시지를 띄우고 다시 입력받습니다.
4. 응답이 OK₩n이면 정상적으로 입차 처리 (UART0_send_string("1234₩n");와 같이 다시 차량 번호 전송)를 진행합니다.

**마스터 코드에 추가될 로직:**

1. 슬레이브 A로부터 [차량번호],C₩n 형태의 메시지를 수신합니다.
2. 수신된 차량 번호로 is_duplicate() 함수를 호출합니다.
3. 결과에 따라 UART0_send_string("OK₩n"); 또는 UART0_send_string("DUP₩n");을 슬레이브 A로 응답합니다.

결론

슬레이브 A에서는 차량 번호를 마스터에게 보내 "이거 중복이니?"라고 물어보고, 마스터의 응답에 따라 입차를 진행할지 결정하는 방식으로 구현해야 합니다.

스마트 주차 관리 시스템

---

Ⅰ. 명세서 (Requirement Specification)

1. 프로젝트 개요 (Overview)

| 항목 | 내용 |
|---|---|
| 1.1 프로젝트명 | 스마트 주차 정산 시스템 |
| 1.2 배경 및 목적 | 다수의 주차장 데이터를 통합 관리하고, 실시간 주차 현황과 요금 정산을 자동화하여 편의성과 효율성을 높이기 위함 |
| 1.3 범위 | 두 개의 주차장(보드 2개)을 슬레이브로 하여 입출차 상태를 감지하고, 마스터 보드에서 입출차 기록 및 요금 정산을 수행함 |

2. 시스템 요구사항

2.1 기능 요구사항 (Functional)

1. 차량의 입차 및 출차 감지 기능 (초음파 센서를 통해)

1. 차량 ID 또는 고유번호 입력 및 인식

1. 입출차 시간 기록 및 주차 요금 정산 기능

1. LCD를 통한 실시간 주차 현황 표시

1. 마스터-슬레이브 간 UART 기반 통신

2.2 비기능 요구사항 (Non-Functional)

- 성능: 응답 시간 ≤ 300 ms

- 신뢰성: 통신 실패 시 재시도 3회 후 오류 표시

- 보안: 차량 ID는 EEPROM에 암호화 저장 가능하도록 구조 고려

- 확장성: 보드 수가 늘어나도 통신 프로토콜 변경 없이 확장 가능

3. 시스템 구성도 및 아키텍처

- 하드웨어 구성
- • ATmega128 × 3, 초음파 센서 × 2, LCD × 3, Keypad × 2, LED x 8, 7-segment display x 2

- 소프트웨어 계층
- • 응용 층 ↔ 통신 층 ↔ 드라이버 층

블록 다이어그램

[Master MCU]?UART?[Slave A MCU]

?UART?[Slave B MCU]

4. 인터페이스 정의 (Interface Specification)

| 구분 | 항목 | 규격 |
|---|---|---|
| UART0 | Baud Rate | 9600 bps |
| | 데이터 포맷 | 8-N-1 |
| | 프로토콜 | STX - [paking_state] - ETX |
| | RX | PE0 |
| | TX | PE1 |
| UART1 | Baud Rate | 9600 bps |
| | 데이터 포맷 | 8-N-1 |
| | 프로토콜 | STX - [car_num & in_time] - ETX |
| | RX | PD0 |
| | TX | PD1 |
| 센서 | 초음파 TRIG | PD2 |
| | 초음파 ECHO | PD3 |
| 센서 | 초음파 TRIG | PE2 |
| | 초음파 ECHO | PE3 |
| LCD | PORTx | 모드 제어 |
| Master | PORTA0~7 | PORTG0~2 |

Slave_A PORTA0~7 PORTG0~2

Slave_B PORTA0~7 PORTG0~2

Keypad DATA 모드 제어

Master PORTC4~7 PORTC0~3

Slave_A PORTF4~7 PORTF0~3

Slave_B PORTC4~7 PORTC0~3

LED PORTx
Slave_A PORTB0~7

7-segment display PORTx 모드 제어
Slave_A PORTD0~7 PORC0~4
Slave_B PORTB0~7 PORTF0~4

5. 제약사항 (Constraints)

- MCU Flash 메모리 8 KB 이내 사용

- 외부 전원 5 V ± 0.2 V

- LCD는 16x2 사용, 문자 오버플로우 방지 고려

6. 테스트 계획 요약

| 테스트 ID | 기능 | 입력 | 기대 결과 |
|---|---|---|---|
| T-01 | 입차 감지 | 차량 모형 투입 | LCD에 “CAR DETECTED" 표시 |

| | | | |
|---|---|---|---|
| T-02 | 출차 및 요금 정산 | 차량 ID OUT 입력 | 차량번호 및 입출차 시간, 요금 표시 |
| T-03 | 주차 현황 표시 | 입출차 반복 | 빈 자리 수 LCD 실시간 업데이트 |

7. 용어 정의 (Glossary)

| 용어 | 정의 |
|---|---|
| ID | 차량 번호 또는 고유 식별자 |
| IN/OUT | 입차/출차 이벤트 구분 |
| Slave A/B | 입차 담당 Slave A, 출차 담당 Slave B로 명명 |
| Master | 차량번호, 입출차 시간 저장, 요금 정산 및 주차공간 업데이트를 담당하는 중앙 처리 보드 |

---

Ⅱ. 유스케이스 (Use-Case)

UC-01 차량 입차 처리

| 항목 | 내용 |
|---|---|
| Use Case 이름 | UC-01 차량 입차 처리 |
| 주요 액터 | 차량 사용자, Slave A MCU, Master MCU |
| 사전 조건 | 시스템 부팅 완료, 센서 대기 상태 |
| 후조건 | 입차 기록 저장, LCD 업데이트 |

| | |
|---|---|
| 기본 흐름 | 1) 초음파 센서 입차 차량 감지 → 2) 차량 ID 입력 (Keypad) → 3) Slave A→ Master 차량번호 전송 → 4) 차량별 시간 카운트 |
| 예외 흐름 | 차량 ID 중복 또는 입력 누락 시 재입력 요청 표시 |
| 관련 시스템 | 초음파 센서, LCD, UART 통신, Master RTC 기능 |

UC-02 차량 출차 처리

| 항목 | 내용 |
|---|---|
| Use Case 이름 | UC-02 차량 출차 처리 |
| 주요 액터 | 차량 사용자, Slave B MCU, Master MCU |
| 사전 조건 | 차량 ID가 이미 시스템에 등록되어 있어야 함 |
| 후조건 | 출차 시간 기록, 주차 요금 계산 및 표시 |
| 기본 흐름 | 1) 초음파 센서 출차 차량 감지 → 2) 차량 ID 입력 (Keypad) → 3) Enter 버튼 누르면 요금정산 완료로 간주 → 4) 주차공간 업데이트 |
| 예외 흐름 | ID 불일치 시 LCD에 오류 메시지 표시<br>출차 시 요금정산 과정에서 일정 시간 경과하면 time out 경고 후 초기화 |
| 관련 시스템 | RTC, LCD, UART, EEPROM |

UC-03 주차 현황 표시

| 항목 | 내용 |
|---|---|
| Use Case 이름 | UC-03 주차 현황 표시 |
| 주요 액터 | 사용자, Slave A MCU, Master MCU |
| 사전 조건 | 시스템 정상 구동 중 |
| 후조건 | Slave A LED로 주차공간 표시 (ON : available, OFF: parked)<br>Slave A, Master MCU에서 상태 업데이트 시, LCD패널에 일정 시간동안 현재 빈 자리/주차된 자리 수 표시 (최대 8대) |

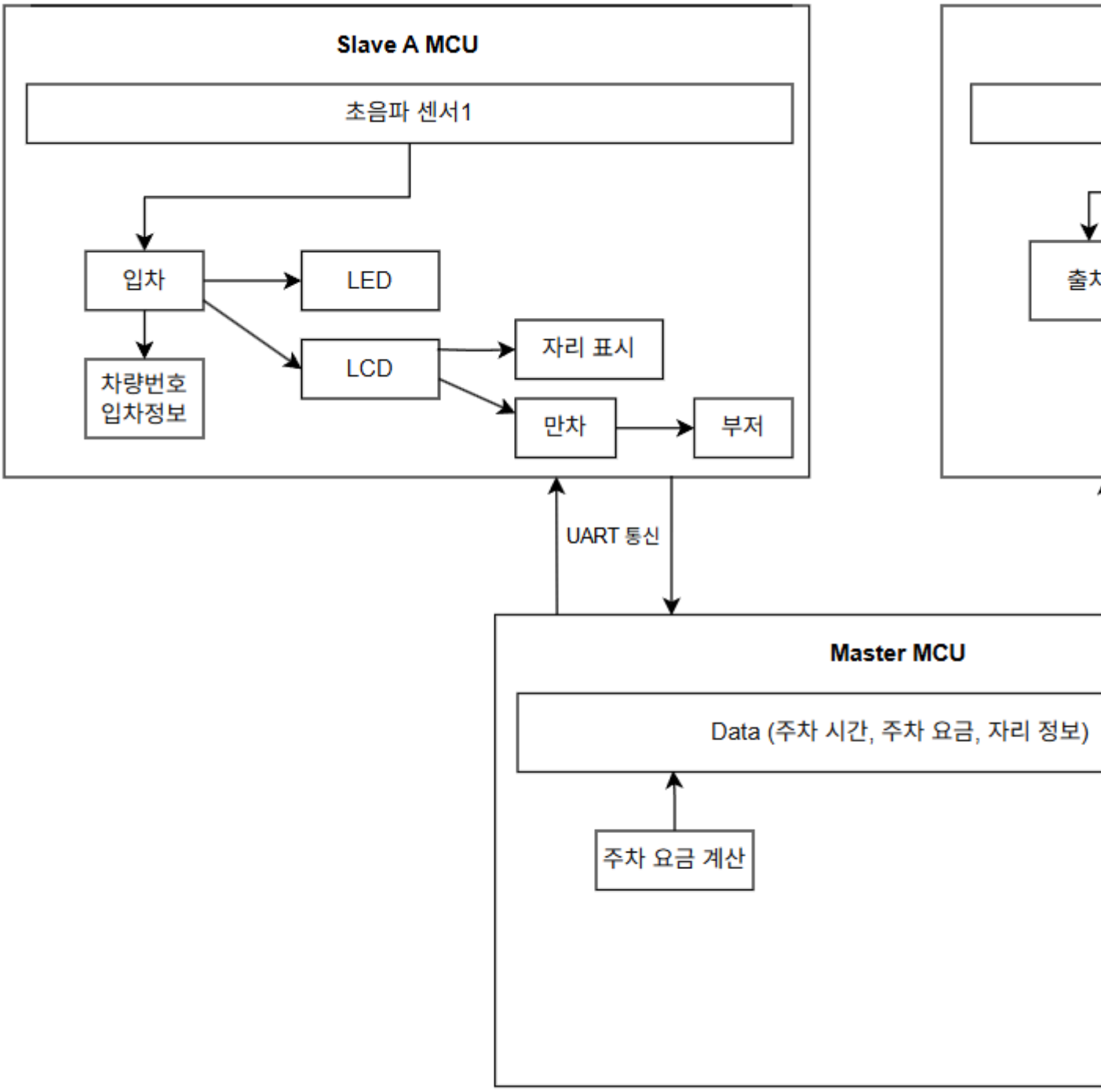
| | |
|---|---|
| 기본 흐름 | 1) 입출차 감지 이벤트 발생 시 → 2) 주차 공간 수 계산 → 3) LCD 업데이트 |
| 예외 흐름 | 센서 고장 시 오류 메시지 표시 |
| 관련 시스템 | LCD, LED |

UC-04 요금 정산

| 항목 | 내용 |
|---|---|
| Use Case 이름 | UC-04 요금 정산 |
| 주요 액터 | Master MCU, 사용자 |
| 사전 조건 | 출차 이벤트 발생, ID가 Master Memory에 기록되어 있어야 함<br>Timer Interrupt로 시간 계산 필요 |
| 후조건 | 출차 - 입차 시간 계산하여 초당 요금 부과 후 LCD에 금액 표시됨 |
| 기본 흐름 | 1) 출차 시 ID 확인 → 2) 입출차 시간 계산 → 3) 요금 계산 → 4) LCD 표시 및 기록 |
| 예외 흐름 | 시간 계산 실패 또는 RTC 오류 시 기본 요금 부과 |
| 관련 시스템 | RTC, EEPROM, LCD, UART |

---

III. Block Diagram



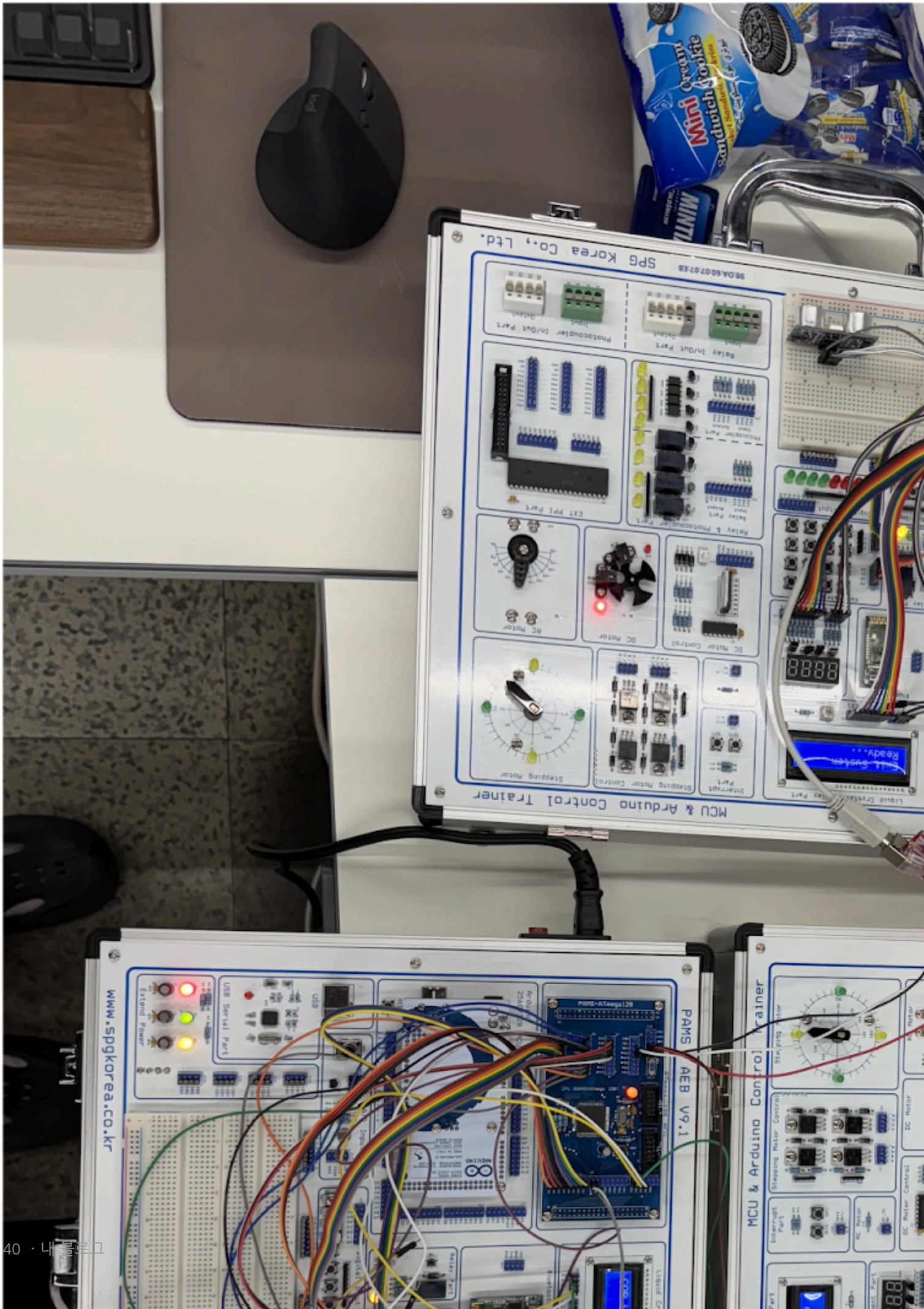

**Block Diagram**

IV. 통신 프로토콜

- 플래그 사용: 양쪽 슬레이브(A, B) 모두 마스터로부터 데이터 수신이 완료되면 parking_state_updated (exit_done 등)와 같은 플래그를 설정하여 main 루프에 데이터 처리 준비가 되었음을 알림.
- ₩n (개행 문자): 슬레이브 A가 마스터로부터 숫자 데이터 수신 완료를 인식하는 데 사용됨. 이는 데이터의 프레임(frame)을 구분하는 역할을 수행.
- 고정 길이: 슬레이브 A가 마스터로 차량 번호를 보낼 때 4자리로 길이가 고정되어 있어, 마스터는 4문자를 받으면 차량 번호 수신이 완료되었음을 알 수 있음.
- 마스터 → 슬레이브 A:
    - parking_state (남은 자리 수): uint8_t 타입의 정수를 ASCII 문자('0' ~ '8')로 변환하여 전송. (예: 7자리가 남았으면 '7'이라는 문자 전송)
    - 종료 문자: 숫자 데이터 전송 후에는 항상 **개행 문자(₩n, ASCII 10)**를 추가로 전송. 이는 슬레이브 A가 숫자 데이터 수신이 완료되었음을 인식하는 신호로 사용됨.

- 슬레이브 A → 마스터: (입차 시)
    - 차량 번호: 사용자가 키패드로 입력한 4자리의 차량 번호를 ASCII 문자 그대로 순서대로 전송합니다. (예: 1234를 입력하면 '1', '2', '3', '4' 순으로 전송)
    - 마스터가 4글자를 받으면 완료로 간주.
- 슬레이브 B → 마스터:
    - [4자리 차량 번호]₩n (ASCII 문자열): 사용자가 입력한 차량 번호에 대한 주차 시간 및 요금 정보를 요청하고, 해당 차량이 출차할 준비가 되었음을 알리게 됨. 사용자가 1234를 입력하고 'E' 버튼을 누르면, 슬레이브 B는 마스터에게 1234₩n (즉, 문자 '1', '2', '3', '4', '₩n' 순으로)을 전송함.
    - [4자리 차량 번호] [,] [D]₩n (ASCII 값 68): 사용자의 결제가 완료되었음을 마스터에게 알리고, 해당 차량에 대한 최종 출차 처리를 요청. 이전에 입력받은 4개의 ASCII 숫자 문자 (예: '1', '2', '3', '4') 및 차량 번호와 출차 신호를 구분하는 쉼표(,) D 값을 전송함. 마스터는 이 'D'를 받아 exit_done 플래그를 설정. 해당 차량의 정보(parked_cars 배열)를 삭제하고 parking_state를 업데이트 하게 됨.
- 마스터 → 슬레이브 B:
    - [주차 시간],[요금]₩n (ASCII 문자열, 쉼표로 구분): 슬레이브 B로부터 요청받은 차량 번호에 대한 주차 시간과 요금 정보를 전달함. 마스터가 120분, 5000원을 전송할 때, 슬레이브 B는 120,5000₩n (즉, 문자 '1', '2', '0', ',', '5', '0', '0', '0', '₩n' 순으로)을 수신함.

V. 회로 구성


MCU & Arduino Control Trainer
SPG Korea Co., Ltd.
PAMS AEB V9.1
www.spgkorea.co.kr
MCU & Arduino Control Trainer

12시 방향 : Slave B MCU (출차 담당)

7시 방향 : Master MCU (중앙 처리 담당)

5시 방향 : Slave A MCU (입차 담당)

VI. System Function Enhancement

1. Error Check : 패리티 비트 추가
2. DC Motor 로 차단기 구현
3. 중복된 차량 체크 로직 추가
4. 정기 주차 차량 요금 할인 / 특정 차량 요금 면제 기능 추가
5. 특정 시간대 요금 할인 또는 할증 기능 추가
6. 에러코드 및 재시도 로직 추가: 통신 실패 시(No master resp.) 단순히 메시지를 띄우는 것 외에, 마스터에게 재요청을 보내거나 특정 횟수만큼 재시도하는 로직을 추가하여 시스템의 안정성 강화.
7. I2C와 같은 다른 통신방식으로 구현